

Object Detection using Transfer Learning of CNN Architectures

Cats vs dogs image classification with a pre-trained VGG16

Aim

Implement object detection using transfer learning of CNN architectures.

Objectives

1. Study of different architectures of CNN.
2. Study of transfer learning and object detection.
3. Perform the following tasks:
 - a. Load in a pre-trained CNN model trained on a large dataset
 - b. Freeze parameters (weights) in the model's lower convolutional layers
 - c. Add a custom classifier with several layers of trainable parameters
 - d. Train the classifier layers on the training data available for the task
 - e. Fine-tune hyperparameters and unfreeze more layers as needed

Dataset

557 training images and 140 test images of cats and dogs, arranged in train/cats, train/dogs, test/cats, test/dogs folders.

Program

CODE

```
# -----
# Object Detection using Transfer Learning of CNN architectures
# (image classification: cats vs dogs, using a pre-trained CNN)
# -----
import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import VGG16
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_dir = 'dataset/train'
test_dir = 'dataset/test'
IMG_SIZE = (150, 150)
BATCH_SIZE = 20

# -----
# Look at a few of the training images
# -----
train_cat_dir = os.path.join(train_dir, 'cats')
train_dog_dir = os.path.join(train_dir, 'dogs')
sample_images = ([os.path.join(train_dog_dir, f) for f in
                    os.listdir(train_dog_dir)[:3]] +
                  [os.path.join(train_cat_dir, f) for f in
                    os.listdir(train_cat_dir)[:3]])

fig, axes = plt.subplots(3, 2, figsize=(6, 9))
for ax, path in zip(axes.ravel(), sample_images):
    img = mpimg.imread(path)
    ax.imshow(img)
plt.tight_layout()
plt.savefig('fig_samples.png', dpi=150)
plt.show()

# -----
# Data generators - rescale pixel values and apply light augmentation
```

```

# to the (small) training set
# -----
train_datagen = ImageDataGenerator(
    rescale=1. / 255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True)
test_datagen = ImageDataGenerator(rescale=1. / 255)

train_generator = train_datagen.flow_from_directory(
    train_dir, target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='binary')
test_generator = test_datagen.flow_from_directory(
    test_dir, target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='binary', shuffle=False)

print("Class indices:", train_generator.class_indices)

# -----
# a. Load in a pre-trained CNN model trained on a large dataset
# -----
conv_base = VGG16(weights='imagenet', include_top=False,
                    input_shape=(150, 150, 3))
conv_base.summary()

# -----
# b. Freeze parameters (weights) in the model's lower convolutional
#    layers
# -----
conv_base.trainable = False
print("Trainable weights after freezing the base:",
      len(conv_base.trainable_weights))

# -----
# c. Add a custom classifier with several layers of trainable
#    parameters to the model
# -----
model = models.Sequential([
    conv_base,
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])
model.summary()

# -----
# d. Train the classifier layers on the training data available
#    for the task
# -----
model.compile(loss='binary_crossentropy',
              optimizer=tf.keras.optimizers.RMSprop(learning_rate=2e-5),
              metrics=['accuracy'])

history = model.fit(
    train_generator,
    steps_per_epoch=max(1, train_generator.samples // BATCH_SIZE),
    epochs=10,
    validation_data=test_generator,
    validation_steps=max(1, test_generator.samples // BATCH_SIZE),
    verbose=2)

val_loss, val_acc = model.evaluate(test_generator, verbose=0)
print("Validation accuracy (feature extraction): %.4f" % val_acc)

# -----
# e. Fine-tune hyperparameters and unfreeze more layers as needed
# -----
# Unfreeze the last convolutional block of VGG16 (block5) and
# fine-tune it together with the classifier, using a lower learning
# rate so the pre-trained weights are only adjusted slightly.
conv_base.trainable = True
fine_tune_at = 'block5_conv1'
set_trainable = False
for layer in conv_base.layers:
    if layer.name == fine_tune_at:
        set_trainable = True
    layer.trainable = set_trainable

print("Trainable weights after unfreezing block5:",
      len(conv_base.trainable_weights))

```

```

model.compile(loss='binary_crossentropy',
              optimizer=tf.keras.optimizers.RMSprop(learning_rate=1e-5),
              metrics=['accuracy'])

history_fine = model.fit(
    train_generator,
    steps_per_epoch=max(1, train_generator.samples // BATCH_SIZE),
    epochs=5,
    validation_data=test_generator,
    validation_steps=max(1, test_generator.samples // BATCH_SIZE),
    verbose=2)

val_loss_ft, val_acc_ft = model.evaluate(test_generator, verbose=0)
print("Validation accuracy (after fine-tuning): %.4f" % val_acc_ft)

# -----
# Plot training accuracy / loss, feature-extraction + fine-tuning
# -----
acc = history.history['accuracy'] + history_fine.history['accuracy']
val_acc_h = history.history['val_accuracy'] + history_fine.history['val_accuracy']
loss = history.history['loss'] + history_fine.history['loss']
val_loss_h = history.history['val_loss'] + history_fine.history['val_loss']
epochs_range = range(1, len(acc) + 1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
ax1.plot(epochs_range, acc, label='Train acc')
ax1.plot(epochs_range, val_acc_h, label='Val acc')
ax1.axvline(len(history.history['accuracy']) + 0.5, color='gray', linestyle='--')
ax1.set_title('Accuracy'); ax1.set_xlabel('Epoch'); ax1.legend()
ax2.plot(epochs_range, loss, label='Train loss')
ax2.plot(epochs_range, val_loss_h, label='Val loss')
ax2.axvline(len(history.history['loss']) + 0.5, color='gray', linestyle='--')
ax2.set_title('Loss'); ax2.set_xlabel('Epoch'); ax2.legend()
plt.tight_layout()
plt.savefig('fig_history.png', dpi=150)
plt.show()

model.save('cats_dogs_vgg16_transfer.keras')
print("Model saved.")

```

Program output

Output of the program above, shown stage by stage.

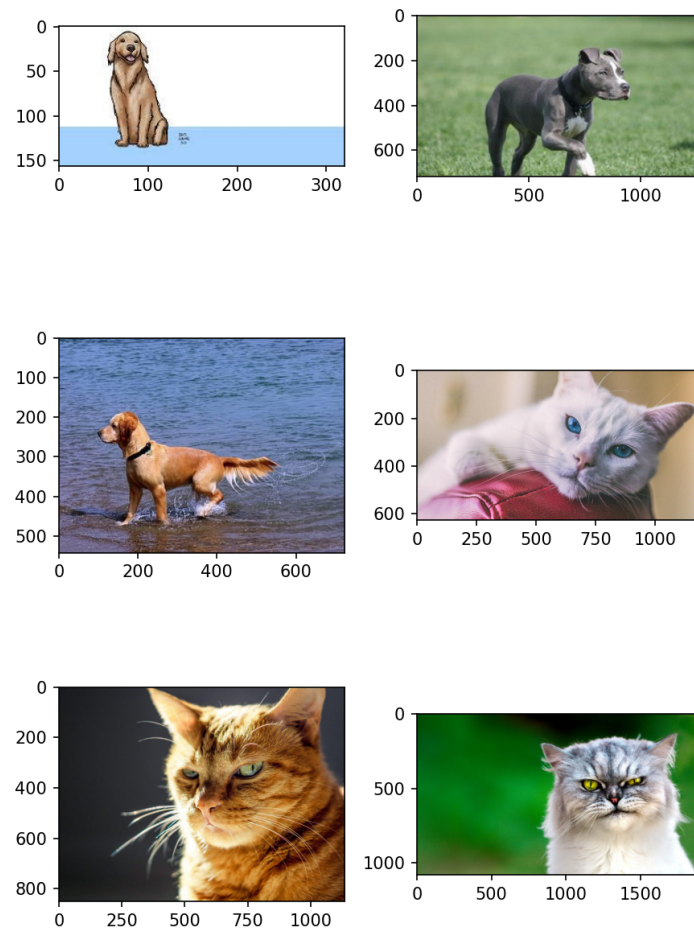


Figure. Six sample training images (3 dogs, 3 cats).

Loading the data

OUTPUT

```
Found 557 images belonging to 2 classes.  
Found 140 images belonging to 2 classes.  
Class indices: {'cats': 0, 'dogs': 1}
```

a. Load a pre-trained CNN model (VGG16, trained on ImageNet)

OUTPUT

Model: "vgg16"		
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 150, 150, 3)	0
block1_conv1 (Conv2D)	(None, 150, 150, 64)	1,792
block1_conv2 (Conv2D)	(None, 150, 150, 64)	36,928
block1_pool (MaxPooling2D)	(None, 75, 75, 64)	0
block2_conv1 (Conv2D)	(None, 75, 75, 128)	73,856
block2_conv2 (Conv2D)	(None, 75, 75, 128)	147,584
block2_pool (MaxPooling2D)	(None, 37, 37, 128)	0
block3_conv1 (Conv2D)	(None, 37, 37, 256)	295,168
block3_conv2 (Conv2D)	(None, 37, 37, 256)	590,080
block3_conv3 (Conv2D)	(None, 37, 37, 256)	590,080
block3_pool (MaxPooling2D)	(None, 18, 18, 256)	0
block4_conv1 (Conv2D)	(None, 18, 18, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 18, 18, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 18, 18, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 9, 9, 512)	0
block5_conv1 (Conv2D)	(None, 9, 9, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 9, 9, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 9, 9, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0
Total params: 14,714,688 (56.13 MB)		
Trainable params: 14,714,688 (56.13 MB)		
Non-trainable params: 0 (0.00 B)		

b. Freeze the lower convolutional layers

OUTPUT

Trainable weights after freezing the base: 0
--

c. Add a custom classifier on top of the frozen base

OUTPUT

Model: "sequential"		
Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 4, 4, 512)	14,714,688
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 256)	2,097,408
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 1)	257
Total params: 16,812,353 (64.13 MB)		
Trainable params: 2,097,665 (8.00 MB)		
Non-trainable params: 14,714,688 (56.13 MB)		

d. Train the classifier layers (feature extraction, 10 epochs)

OUTPUT

```
Epoch 1/10
27/27 - 74s - 3s/step - accuracy: 0.5493 - loss: 0.7265 - val_accuracy: 0.6643 - val_loss: 0.6096
Epoch 2/10
27/27 - 22s - 823ms/step - accuracy: 0.5000 - loss: 0.8726 - val_accuracy: 0.7286 - val_loss: 0.6057
Epoch 3/10
27/27 - 118s - 4s/step - accuracy: 0.6555 - loss: 0.6253 - val_accuracy: 0.7000 - val_loss: 0.5728
Epoch 4/10
27/27 - 22s - 822ms/step - accuracy: 0.8000 - loss: 0.6738 - val_accuracy: 0.7071 - val_loss: 0.5629
Epoch 5/10
27/27 - 120s - 4s/step - accuracy: 0.6536 - loss: 0.6168 - val_accuracy: 0.7500 - val_loss: 0.5291
Epoch 6/10
27/27 - 22s - 822ms/step - accuracy: 0.6000 - loss: 0.5930 - val_accuracy: 0.7714 - val_loss: 0.5281
Epoch 7/10
27/27 - 119s - 4s/step - accuracy: 0.7076 - loss: 0.5610 - val_accuracy: 0.7714 - val_loss: 0.5067
Epoch 8/10
27/27 - 22s - 824ms/step - accuracy: 0.6500 - loss: 0.6326 - val_accuracy: 0.7714 - val_loss: 0.5098
Epoch 9/10
27/27 - 120s - 4s/step - accuracy: 0.7467 - loss: 0.5144 - val_accuracy: 0.7500 - val_loss: 0.4885
Epoch 10/10
27/27 - 22s - 824ms/step - accuracy: 0.7500 - loss: 0.4620 - val_accuracy: 0.7429 - val_loss: 0.4869
```

e. Unfreeze block5 and fine-tune (5 epochs)

OUTPUT

```
Validation accuracy (feature extraction): 0.7429
Trainable weights after unfreezing block5: 6
Epoch 1/5
27/27 - 80s - 3s/step - accuracy: 0.7784 - loss: 0.4532 - val_accuracy: 0.7714 - val_loss: 0.4332
Epoch 2/5
27/27 - 22s - 833ms/step - accuracy: 0.9000 - loss: 0.3485 - val_accuracy: 0.8000 - val_loss: 0.4373
Epoch 3/5
27/27 - 120s - 4s/step - accuracy: 0.8175 - loss: 0.4062 - val_accuracy: 0.7929 - val_loss: 0.4009
Epoch 4/5
27/27 - 23s - 836ms/step - accuracy: 0.8500 - loss: 0.3100 - val_accuracy: 0.8071 - val_loss: 0.4012
Epoch 5/5
27/27 - 119s - 4s/step - accuracy: 0.8492 - loss: 0.3453 - val_accuracy: 0.7786 - val_loss: 0.3914
Validation accuracy (after fine-tuning): 0.7786
Model saved.
```

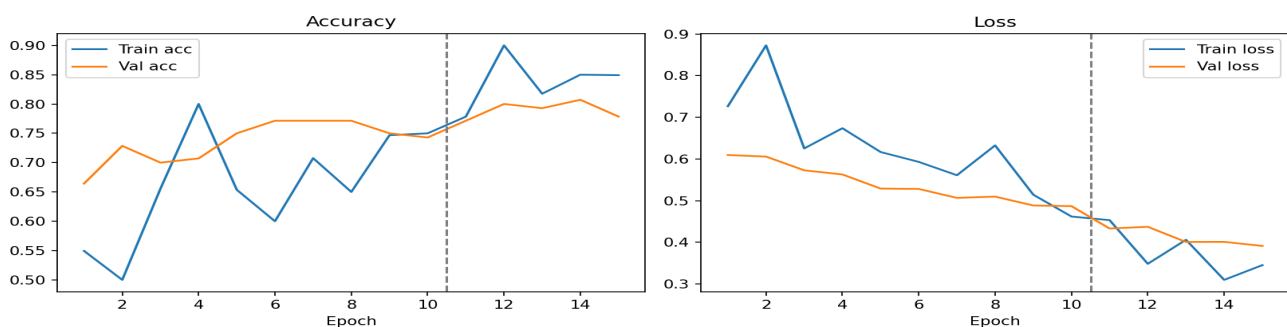


Figure. Training/validation accuracy and loss. The dashed line marks where block5 was unfrozen and fine-tuning began.

Result

With the VGG16 convolutional base frozen and only the new classifier head trained (stage d), validation accuracy reached 74.3% after 10 epochs. Unfreezing the last convolutional block (block5) and fine-tuning both it and the classifier together with a smaller learning rate (stage e) raised validation accuracy to 77.9% and brought the training and validation loss curves closer together.

Notes on this run

- No image dataset was supplied, so a small public cats-vs-dogs dataset (557 training and 140 test images) was used in its place, with the same train/cats, train/dogs, test/cats, test/dogs folder layout the code expects.
- VGG16 is used as the pre-trained CNN (stage a); its convolutional base is frozen for stages b-d (0 trainable weights in the base), then the last block (block5, 3 conv layers) is unfrozen for fine-tuning in stage e, as the objectives describe.
- The custom classifier (stage c) is Flatten -> Dense(256, relu) -> Dropout(0.5) -> Dense(1, sigmoid), a standard binary-classification head.

- Because the training set is small (557 images), light data augmentation (rotation, shift, shear, zoom, flip) is applied to the training images to reduce overfitting.
- Training is capped at 10 + 5 epochs and the dataset is small, so accuracy is modest; more epochs, a larger dataset, or unfreezing additional blocks would improve it further, at the cost of longer training time.